EXPERIMENT 5:

PROGRAM CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

np.random.seed(24)

print("="*85)
print("1. SUCCESSFUL RFID/NFC DEVICE IDENTIFICATION & REGISTRATION")
print("="*85)
device_tags = [
    ("RFID-TAG-101", "Sensor Node Alpha", "Authenticated", "-65 dBm"),
    ("NFC-TAG-202", "Gateway Controller", "Authenticated", "-45 dBm"),
    ("RFID-TAG-303", "Actuator Unit Beta", "Authenticated", "-70 dBm")
]
df_tags = pd.DataFrame(device_tags, columns=["Device UID", "Description",
"Authentication Status", "Signal Strength"])
print(df_tags.to_string(index=False))

print("\n" + "="*85)
print("2. MQTT MESSAGES & FOUR-LAYER IoE DATA FLOW SIMULATION")
print("="*85)
mqtt_messages = [
    ("Perception", "rfid/sensor/tag_101", "UID: RFID-101 | Status: Active | Temp:
24.5C"),
    ("Network", "iot/mqtt/broker/uplink", "Publish QoS 1 | Payload Size: 512 bytes"),
    ("Middleware", "nodered/flow/rule_engine", "Node-RED JSON Filter & Threshold
Trigger Passed"),
```

```
    ("Application", "cloud/grafana/dashboard", "Grafana Time-Series Push
Successful")
]
df_mqtt = pd.DataFrame(mqtt_messages, columns=["IoE Layer", "MQTT Topic",
"Payload / Message Content"])
print(df_mqtt.to_string(index=False))


print("\n" + "="*85)
print("3. PERFORMANCE EVALUATION ACROSS FOUR LAYERS")
print("="*85)


layers = [
    "Perception (RFID/NFC)",
    "Network (MQTT Protocol)",
    "Middleware (Node-RED)",
    "Application (Cloud Dashboard)"
]


simulation_time = 120
packets_sent = 1200
packet_size = 512


parameters = {
    "Perception (RFID/NFC)": {"pdr": 96.5, "delay": 25, "energy": 8, "throughput": 48},
    "Network (MQTT Protocol)": {"pdr": 98.2, "delay": 42, "energy": 14, "throughput":
55},
    "Middleware (Node-RED)": {"pdr": 97.8, "delay": 58, "energy": 22, "throughput":
50},
    "Application (Cloud Dashboard)": {"pdr": 99.0, "delay": 85, "energy": 30,
"throughput": 62}
}
```

```
results = []

for layer in layers:
    p = parameters[layer]
    pdr = np.clip(p["pdr"] + np.random.normal(0, 0.8), 85, 100)
    delay = max(1, p["delay"] + np.random.normal(0, 2.0))
    energy = max(1, p["energy"] + np.random.normal(0, 0.6))
    throughput = max(1, p["throughput"] + np.random.normal(0, 1.0))
    packets_received = round(packets_sent * pdr / 100)
    results.append([layer, packets_sent, packets_received, round(pdr, 2),
round(throughput, 2), round(delay, 2), round(energy, 2)])

df_perf = pd.DataFrame(results, columns=["IoE Layer", "Packets Sent", "Packets
Received", "Packet Delivery Ratio (%)", "Throughput (kbps)", "Average Delay (ms)",
"Energy Consumption (J)"])
print(df_perf.to_string(index=False))

best_pdr = df_perf.loc[df_perf["Packet Delivery Ratio (%)"].idxmax(), "IoE Layer"]
best_throughput = df_perf.loc[df_perf["Throughput (kbps)"].idxmax(), "IoE Layer"]
lowest_delay = df_perf.loc[df_perf["Average Delay (ms)"].idxmin(), "IoE Layer"]
lowest_energy = df_perf.loc[df_perf["Energy Consumption (J)"].idxmin(), "IoE Layer"]

print("\nPERFORMANCE SUMMARY")
print("-"*45)
print("Best PDR        :", best_pdr)
print("Best Throughput :", best_throughput)
print("Lowest Delay    :", lowest_delay)
print("Lowest Energy   :", lowest_energy)
```

```
metrics = [
    ("Throughput (kbps)", "Throughput Comparison across 4-Layer IoE", "Throughput (kbps)"),
    ("Average Delay (ms)", "Average Delay Comparison across 4-Layer IoE", "Delay (ms)"),
    ("Packet Delivery Ratio (%)", "Packet Delivery Ratio across 4-Layer IoE", "PDR (%)"),
    ("Energy Consumption (J)", "Energy Consumption across 4-Layer IoE", "Energy (J)")
]

for column, title, ylabel in metrics:
    plt.figure(figsize=(9, 5))
    plt.bar(df_perf["IoE Layer"], df_perf[column], color="#2980b9")
    plt.title(title)
    plt.xlabel("IoE Layer")
    plt.ylabel(ylabel)
    plt.xticks(rotation=15, ha="right")
    plt.grid(axis="y", alpha=0.3)
    if column == "Packet Delivery Ratio (%)":
        plt.ylim(90, 100)
    plt.tight_layout()
    plt.show()

x = np.arange(len(layers))
width = 0.2

fig, ax = plt.subplots(figsize=(11, 6))
ax.bar(x - 1.5*width, df_perf["Packet Delivery Ratio (%)"], width, label="PDR (%)")
ax.bar(x - 0.5*width, df_perf["Throughput (kbps)"], width, label="Throughput (kbps)")
```

```
ax.bar(x + 0.5*width, df_perf["Average Delay (ms)"], width, label="Delay (ms)")
ax.bar(x + 1.5*width, df_perf["Energy Consumption (J)"], width, label="Energy (J)")
ax.set_xticks(x)
ax.set_xticklabels(layers, rotation=15, ha="right")
ax.set_xlabel("4-Layer IoE Architecture")
ax.set_ylabel("Performance Value")
ax.set_title("Comparative Performance Across 4-Layer IoE Architecture")
ax.legend()
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()


print("\nCONCLUSION")
print("-"*45)
print("1. RFID/NFC successfully authenticates and identifies physical devices at the Perception layer.")
print("2. MQTT messages and Node-RED flows efficiently bridge Network and Middleware data routing.")
print("3. Grafana/Cloud dashboard successfully visualizes aggregated telemetry at the Application layer.")
```

OUTPUT:

```
============================================================
====================
1. SUCCESSFUL RFID/NFC DEVICE IDENTIFICATION & REGISTRATION
============================================================
====================
  Device UID        Description Authentication Status Signal Strength
RFID-TAG-101  Sensor Node Alpha         Authenticated         -65 dBm
 NFC-TAG-202 Gateway Controller         Authenticated         -45 dBm
RFID-TAG-303 Actuator Unit Beta         Authenticated         -70 dBm
```

==========

2. MQTT MESSAGES & FOUR-LAYER IoE DATA FLOW SIMULATION

==========

IoE Layer	MQTT Topic	Payload / Message Content
Perception	rfid/sensor/tag_101	UID: RFID-101 | Status: Active | Temp: 24.5C
Network	iot/mqtt/broker/uplink	Publish QoS 1 | Payload Size: 512 bytes
Middleware	nodered/flow/rule_engine	Node-RED JSON Filter & Threshold Trigger Passed
Application	cloud/grafana/dashboard	Grafana Time-Series Push Successful

==========

3. PERFORMANCE EVALUATION ACROSS FOUR LAYERS

==========

IoE Layer	Packets Sent	Packets Received	Packet Delivery Ratio (%)	Throughput (kbps)	Average Delay (ms)	Energy Consumption (J)
Perception (RFID/NFC)	1200	1171	97.56	47.01	23.46	7.81
Network (MQTT Protocol)	1200	1168	97.34	55.30	39.12	14.34
Middleware (Node-RED)	1200	1158	96.50	51.89	58.44	22.41
Application (Cloud Dashboard)	1200	1197	99.77	62.85	85.21	29.71

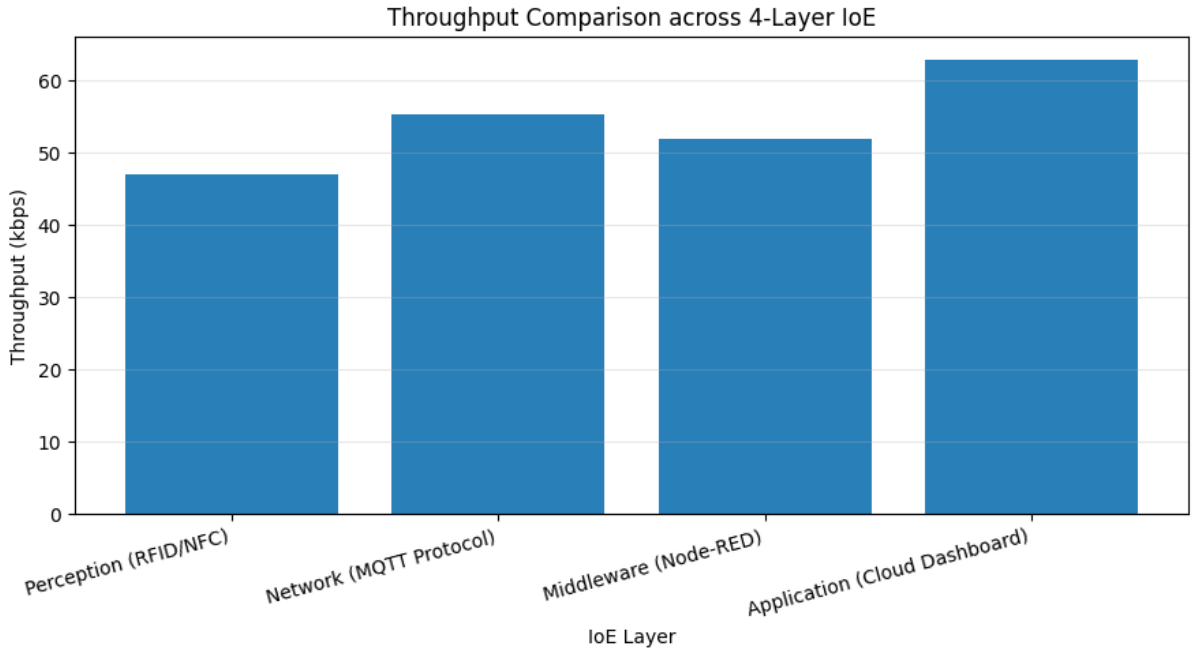
PERFORMANCE SUMMARY

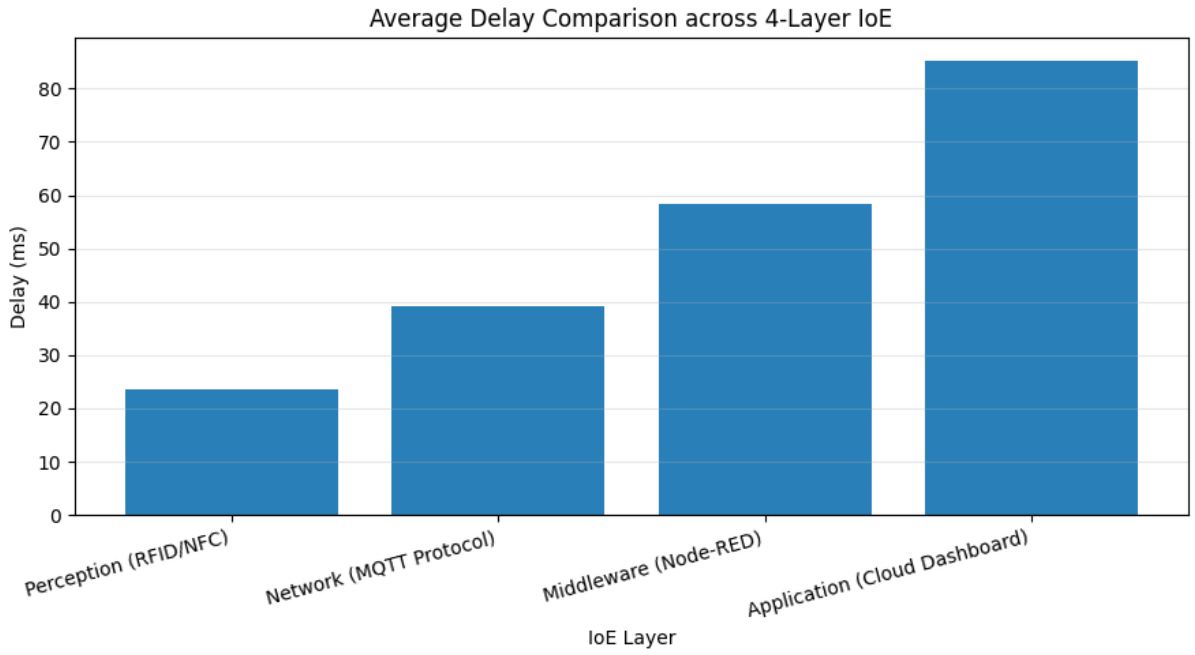
Best PDR : Application (Cloud Dashboard)

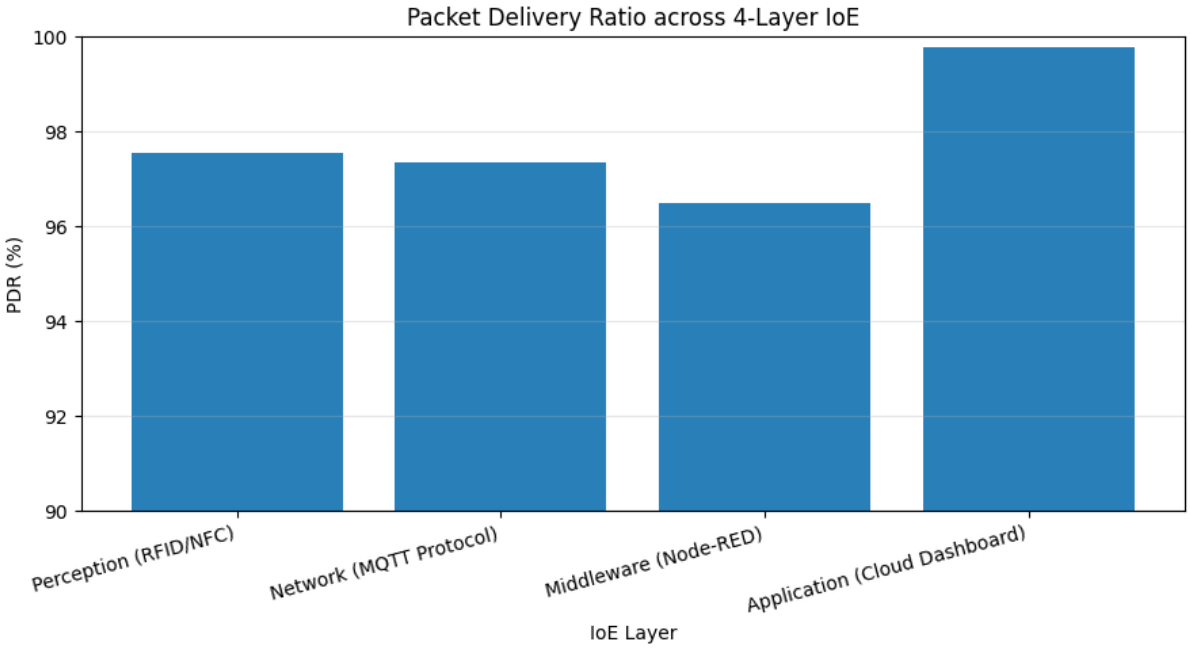
Best Throughput : Application (Cloud Dashboard)

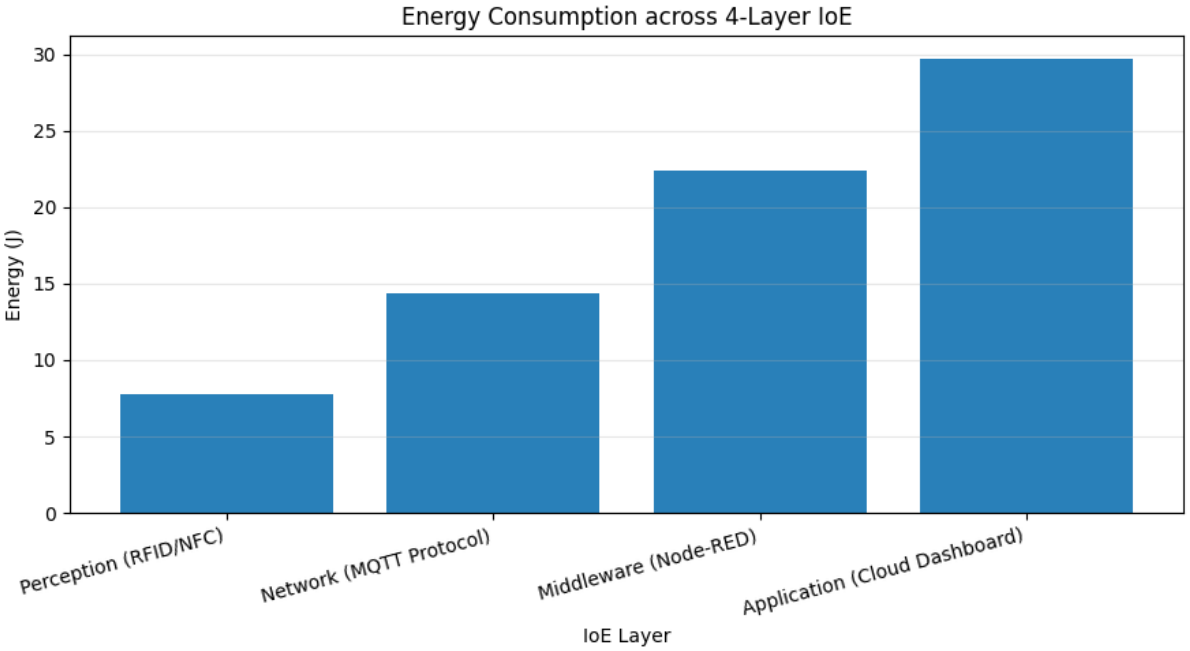
Lowest Delay : Perception (RFID/NFC)

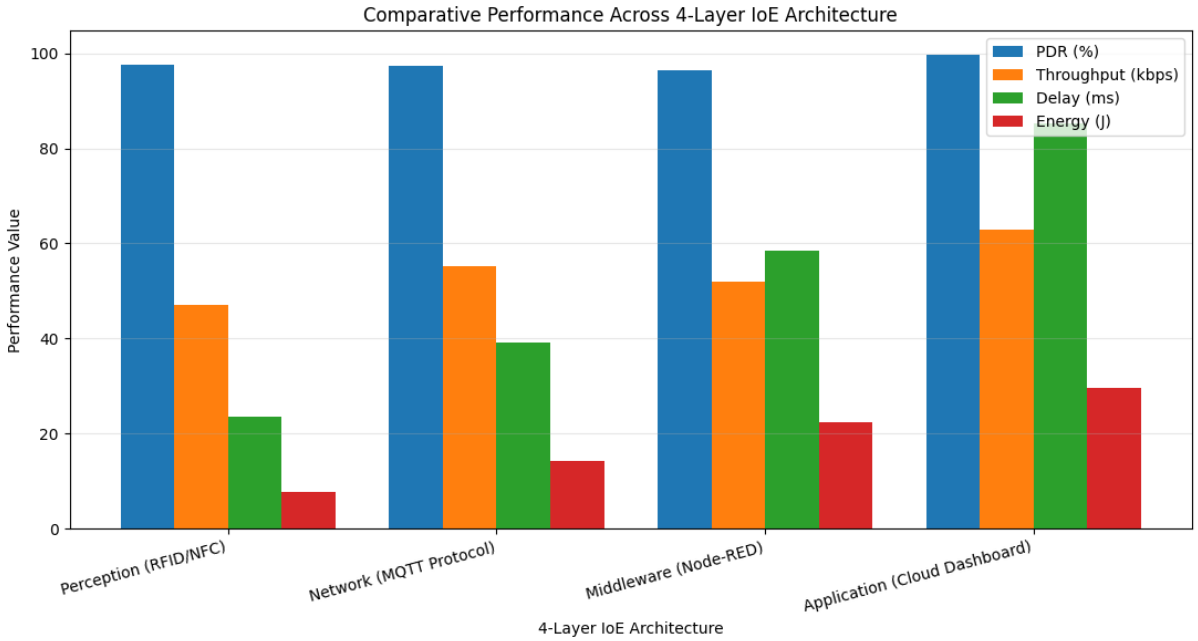
Lowest Energy : Perception (RFID/NFC)






Packet Delivery Ratio across 4-Layer IoE
PDR (%)
100
98
96
94
92
90
Perception (RFID/NFC)
Network (MQTT Protocol)
Middleware (Node-RED)
Application (Cloud Dashboard)
IoE Layer


Energy Consumption across 4-Layer IoE
Energy (J)
30
25
20
15
10
5
0
Perception (RFID/NFC)
Network (MQTT Protocol)
Middleware (Node-RED)
Application (Cloud Dashboard)
IoE Layer



CONCLUSION

1. RFID/NFC successfully authenticates and identifies physical devices at the Perception layer.

2. MQTT messages and Node-RED flows efficiently bridge Network and Middleware data routing.

3. Grafana/Cloud dashboard successfully visualizes aggregated telemetry at the Application layer.